

```
1 #%%
2 from asyncio.windows_events import NULL
3 from operator import index
4
5 import drop
6 import pandas as pd
7 import numpy as np
8 from sklearn.model_selection import
   train_test_split
9 from sklearn.linear_model import LinearRegression
10 #%% md
11 ## 1. Load Data Using Pandas.
12 #%%
13 # Read the dataset from the csv file using the
   pandas library.
14 df = pd.read_csv("googleplaystore.csv")
15
16 #%% md
17 ### 1.1 Check for successful loading of the
   dataset.
18 #%%
19 # Get the first 5 fields and records of the
   dataset.
20 df.head(20)
21 #%%
22 # Get the last 5 fields and records of the
   dataset.
23 df.tail()
24 #%% md
25 ## 2. Check for null values in the data.
26 #%%
27 # Check if there are any 'null' records in any of
   the records belonging to any of the fields.
28 print(df.isnull().sum().sort_values(ascending=
   False))
29 print("Total number of null values: ", df.isnull
   ().sum().sum())
30 #%% md
```

```
31 ## 3. Drop records with null values in any of the  
    columns.  
32 ###  
33 # Drop the null values.  
34 df = df.dropna()  
35 print(df.isnull().sum())  
36 print("Total number of null values: ", df.isnull()  
    ().sum().sum())  
37 ### md  
38 ## 4. Formatting variables into correct data  
    types.  
39 ###  
40 df['Category'] = df['Category'].astype('category'  
    ) #Set category records as category.  
41 df['Reviews'] = df['Reviews'].astype(int) #Set  
    reviews records as integers.  
42 df['Installs'] = df['Installs'].str.replace('+',  
    '').str.replace(',', '').astype(int) #Set  
    installs records as Integers.  
43 df['Price'] = df['Price'].str.replace('$', '').  
    astype(float) #Set price records as float.  
44 df['Content Rating'] = df['Content Rating'].  
    astype('category') #Set content rating as  
    category.  
45 df['Genres'] = df['Genres'].astype('category') #  
    Set genres as category.  
46 df['Last Updated'] = pd.to_datetime(df['Last  
    Updated']) #Set last updated to datetime.  
47 df['Current Ver'] = df['Current Ver'].replace(''  
    Varies with device', np.nan).astype(object) #Set  
    current ver as object.  
48 df['Android Ver'] = df['Android Ver'].replace(''  
    Varies with device', np.nan).astype(object) #Set  
    android ver as object.  
49 df['Rating'] = pd.to_numeric(df['Rating'], errors  
    ='coerce')  
50  
51 ###
```

```

52 #Format the size field and its records.
53 df['Size'] = df['Size'].replace('Varies with
    device', np.nan)
54
55 df['Size'] = df['Size'].apply(lambda x: float(str
    (x).replace('M', '')) if 'M' in str(x)
56                                else float(str(x).
    replace('k','')) / 1000 if 'k' in str(x)
57                                else np.nan)
58 ###
59 df.info()
60 ### md
61 ### 4.1 Sanity Checks.
62 ###
63 print("Minimum rating: ", df['Rating'].min())
64 print("Maximum rating: ", df['Rating'].max())
65 ###
66 print('Number of installs: ', df['Installs'].
    count())
67 print('Number of reviews: ', df['Reviews'].count
    ())
68 print('Number of ratings: ', df['Rating'].count
    ())
69 ###
70 print(df[['Type','Price']].drop_duplicates()) #
    Drop duplicates on the type and price fields.
71 ### md
72 ## 5. Performing univariate analysis
73 ###
74 import seaborn as sns #Import seaborn library.
75 import matplotlib.pyplot as plt #Import matplotlib
    library.
76 ### md
77 ### 5.1 Outlier treatment.
78 ###
79 #Get different quantiles of the install variable.
80 print('10th Quantile: ', df['Installs'].quantile(
    0.1))

```

```
81 print('25th Quantile: ', df['Installs'].quantile
    (0.25))
82 print('50th Quantile: ', df['Installs'].quantile
    (0.50))
83 print('70th Quantile: ', df['Installs'].quantile
    (0.70))
84 print('90th Quantile: ', df['Installs'].quantile
    (0.90))
85 print('99th Quantile: ', df['Installs'].quantile
    (0.99))
86
87 #Filter out records not meeting specific
    conditions.
88 df = df[
89     (df['Price'] <= 200) &
90     (df['Installs'] <= 2000000) &
91     (df['Reviews'] <= 2000000) &
92     (df['Rating'] >= 1) &
93     (df['Rating'] <= 5) &
94     (df['Rating'].notna())
95 ]
96 ### md
97 ### 5.2 Boxplot for price.
98 ###
99 sns.boxplot(x = 'Price', data = df)
100 plt.show()
101 ### md
102 ### 5.3 Boxplot for reviews.
103 ###
104 sns.boxplot(x = 'Reviews', data = df)
105 plt.show()
106 ### md
107 ### 5.4 Histogram for rating.
108 ###
109 sns.histplot(data = df, x = 'Rating')
110 plt.show()
111 ### md
112 ### 5.5 Histogram for size
```

```
113 #%%
114 sns.histplot(data = df, x = 'Size')
115 plt.show()
116 #%% md
117 ## 6. Bivariate analysis
118 #%% md
119 ### 6.1 Scatter plot for Rating vs. Price.
120 #%%
121 sns.scatterplot(data = df, x = 'Rating', y = '
    Price')
122 plt.show()
123 #%% md
124 ### 6.2 Scatter plot for Rating vs. Size.
125 #%%
126 sns.scatterplot(data = df, x = 'Rating', y = '
    Size')
127 plt.show()
128 #%% md
129 ### 6.3 Scatter plot for Rating vs. Reviews.
130 #%%
131 sns.scatterplot(data = df, x = 'Rating', y = '
    Reviews')
132 plt.show()
133 #%% md
134 ### 6.4 Boxplot for Rating vs. Content Rating.
135 #%%
136 sns.boxplot(data = df, x = 'Content Rating', y
    = 'Rating')
137 plt.xticks(rotation = 45)
138 plt.show()
139 #%% md
140 ### 6.5 Boxplot for Rating vs. Category.
141 #%%
142 sns.boxplot(data = df, x = 'Category', y = '
    Rating')
143 plt.xticks(rotation = 90)
144 plt.show()
145 #%% md
```

```
146 ## 7. Data preprocessing and formatting.
147 ###
148 inp1 = df #creating a copy of a dataframe
149 ###
150 print(inp1.head())
151 ###
152 df['Installs_log'] = np.log1p(inp1['Installs'])
    #Reducing the skewness of the Installs records
    using the np.log1p function.
153 print(df['Installs_log'].unique())
154 ###
155 df['Reviews_log'] = np.log1p(inp1['Reviews']) #
    Reducing the skewness of the Reviews records
    using the np.log1p function.
156 print(df['Reviews_log'].unique())
157 ###
158 inp2 = pd.get_dummies(df[['Category', 'Content
    Rating', 'Genres']], drop_first=False).astype(
    int)
159 print(inp2) #One-hot encoding
160 ### md
161 ## 8. Train-test split the dataset.
162 ###
163 df_copy = df[['Installs_log', 'Rating']]
164
165 df_copy_grouped = df_copy.groupby(['Installs_log
    '])['Rating'].mean().reset_index()
166
167 print(df_copy_grouped)
168
169 # Categorical columns
170 cat_cols = df_copy_grouped.select_dtypes(include
    =['category'])
171
172 # Numerical columns
173 num_cols = df_copy_grouped.select_dtypes(include
    =['int64', 'float64'])
174
```

```
175 print("Categorical Columns:", cat_cols.columns)
176
177 print("\nNumerical Columns:", num_cols.columns)
178
179 #cat_encoded = pd.get_dummies(cat_cols,
    drop_first=True)
180
181 #df_final = pd.concat([cat_encoded, num_cols],
    axis=1)
182
183 df_train, df_test = train_test_split(num_cols,
    test_size = 0.3, random_state = 42) #Creating
    train and test dataframes using the
    train_test_split function.
184 print(df_train.shape, df_test.shape)
185


---


186 ###
187
188 X_train = df_train.drop('Rating', axis=1)
189 y_train = df_train['Rating']
190
191 X_test = df_test.drop('Rating', axis=1)
192 y_test = df_test['Rating']
193


---


194 ### md
195 ## 9. Build the model


---


196 ###
197 model = LinearRegression()
198 model.fit(X_train, y_train)
199
200 model_score_train = model.score(X_train, y_train
    )
201 model_score_test = model.score(X_test, y_test)
202
203 print('Training Score: ', round(
    model_score_train,2))
204 print('Test Score: ', round(model_score_test,2))


---


205 ### md
```

```
206 ## 10. Make predictions on the test set.  
207 #%%  
208 y_prediction = model.predict(X_train)  
209 plt.scatter(y_train, y_prediction)  
210 plt.xlabel('Actual Rating')  
211 plt.ylabel('Predicted Rating')  
212 plt.title('Actual vs Predicted Rating')  
213 plt.show()  
214 #%%  
215
```